

Basic (plug search in as a plain tool first) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real tool-calling setup handles the messy edge cases). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

You already built a search function back in Doc08 — `retrieve()` . This exercise doesn't ask you to write new search logic. It asks you to hand that existing function to your Doc09 graph, the same way you'd hand any other tool to an agent.

Think of a "tool" as just a function with a label on it saying "the model is allowed to call this." Once it's labeled, the model can decide to use it while it's thinking, the same way it would use a calculator tool or a weather tool.

Things to use:

- Your existing `retrieve(query)` function, unchanged, from `08_rag` .
- `from langchain_core.tools import tool` — a decorator.
- `@tool` placed on top of a small wrapper function that calls `retrieve()` inside it.
- Your Doc09 graph's model node, which already calls `model.bind_tools([...])` — add your new tool to that same list.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The exercise is really about reusing Doc08's `retrieve()` function inside Doc09's tool-calling machinery, without rewriting either.

`@tool` is what turns a plain Python function into something a model-calling node can offer to the model. The function's type hints and docstring together *are* the tool's description — there's no separate config file to write, and the model reads that docstring, at call time, to decide when to use the tool. A vague docstring gives you a tool the model calls at the wrong times, or never.

The exact pieces:

- `from langchain_core.tools import tool`, then `@tool` on a wrapper: `def search_docs(query: str) -> str: .`
- The docstring is the whole tool description — write it as an instruction to the model, not a comment to yourself (e.g. "Search the document set for text relevant to the query" rather than "wraps retrieve()").
- `results = retrieve(query)` inside the wrapper, then `"\n\n".join(chunk.page_content for chunk in results)` to turn a list of `Document` objects into the plain string the model can actually read.
- `model.bind_tools([search_docs])` — if your Doc09 model node already binds other tools, add `search_docs` to that same list rather than creating a second, separate binding call.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think about what a tool function's return value actually is: it becomes a `ToolMessage` that gets inserted straight into the model's context. That changes two things you haven't had to worry about yet.

First — what happens when `retrieve()` itself fails? A vector store can time out or be briefly unreachable (this is exactly what the `search_failure` exercise practices at the graph-node level) — but a tool is called *from inside* a model turn, not from your own script, so an uncaught exception there doesn't fail as cleanly. A raw traceback landing in the model's context is confusing and wastes tokens; the tool should catch the failure itself and hand back a short, honest string the model can react to sensibly, like "Search is temporarily unavailable."

Second — what stops `search_docs` from returning more text than the model can reasonably use? On a small demo document set, joining every matching chunk is harmless. On a real document set, an unbounded join can eat a large slice of your context window on one tool call, especially if the model calls it more than once in the same turn.

The real design question isn't just "wrap `retrieve()` in `@tool`" — it's "what does this tool hand back when the search itself is unhealthy, or when it technically 'succeeds' but with far more text than is useful?"

The extra pieces needed:

- A `try/except` around the `retrieve()` call inside the tool, returning a plain string like "Search is temporarily unavailable." on failure instead of letting the exception propagate up through the model's tool-calling loop.
- A cap on how much gets joined and returned — either `results[:5]` to cap the number of chunks, or a character-count cap on the joined string — so one tool call can't consume an outsized share of the context window.
- Optionally, recording *which* chunks were used (source file, position) somewhere the graph can see later, not just the joined text — so a downstream node (like the approval step in `approval_pause`) can show a human exactly what was searched, not just a paraphrase.

Sketch this hardened version yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic names the tool concept and the exact pieces for the happy path — wrap, decorate, bind. Intermediate explains why the docstring and return type matter and shows the real syntax. Advanced asks what this tool does when `retrieve()` itself is unhealthy, or when it "works" but returns more than the model should have to read in one turn — the same "don't let a plain function's failure or excess become the caller's

problem silently" idea that shows up again in `search_failure` and `empty_search` , just applied one layer earlier, at the tool boundary.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
import retrieve from your Doc08 code

make a new tool called search_docs that takes a query:
    call retrieve(query)
    turn the results into one plain text string
    return that string

find your Doc09 graph's model node:
    add search_docs to the list of tools it binds

run the graph with a question about your document set:
    check the model actually calls search_docs
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# search_tool_integration_practice.py – Basic section
from langchain_core.tools import tool
from retriever import retrieve

@tool
def search_docs(query):
    """Search the document set for relevant text."""
    results = retrieve(query)
    text = ""
    for chunk in results:
        text = text + chunk.page_content + "\n\n"
    return text
```

Expected output if you run just this (nothing calls it yet): nothing — defining a tool doesn't call it. Bind it with `model.bind_tools([search_docs])`, invoke the graph with a question about your documents, and check the returned messages for a `tool_calls` entry naming `search_docs`.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
from retriever import retrieve  # your Doc08 function

@tool
def search_docs(query: str) -> str:
    """Search the document set for text relevant to the query."""
    results = retrieve(query)
    return "\n\n".join(chunk.page_content for chunk in results)

# in your Doc09 graph's model node:
model_with_tools = model.bind_tools([search_docs])

# run it:
graph.invoke({"messages": [("user", "What does the document say about X?")]}))
```

```
# search_tool_integration_practice.py – Basic section
from langchain_core.tools import tool
from retriever import retrieve

@tool
def search_docs(query: str) -> str:
    """Search the document set for text relevant to the query."""
    results = retrieve(query)
    return "\n\n".join(chunk.page_content for chunk in results)
```

Wire this into `bind_tools([...])`, run the graph, and confirm a `tool_calls` entry naming `search_docs` shows up before comparing against the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
@tool
def search_docs(query: str) -> str:
    try:
        results = retrieve(query)
    except an exception from the search backend:
        return "Search is temporarily unavailable."

    if results is empty:
        return "No relevant documents found."

    capped_results = only the first 5 results
    return capped_results joined into one string, each chunk separated by a blank
```

Here's almost the whole thing — fill in the missing piece yourself:

```
# search_tool_integration_practice.py – Basic section
from langchain_core.tools import tool
from retriever import retrieve

MAX_RESULTS = 5

@tool
def search_docs(query: str) -> str:
    """Search the document set for text relevant to the query."""
    try:
        results = retrieve(query)
    except Exception:
        return "Search is temporarily unavailable."

    if not results:
        return "No relevant documents found."

    # your turn: cap `results` to MAX_RESULTS before joining, so one
    # tool call can't return more text than the model should read at once
    ...

    return "\n\n".join(chunk.page_content for chunk in results)
```

Fill in the cap yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: the same underlying shape (call `retrieve()` , join the text, return a string) at 3 completeness levels — Basic's version trusts `retrieve()` to always succeed and always return a reasonable amount of text, Intermediate adds the type contract and an explicit "no results" message but still trusts the backend not to fail or over-return, and Advanced adds the two checks that only matter once this tool is called from inside a real, unattended agent run — a caught failure instead of a crashed turn, and a cap instead of an unbounded dump into the model's context.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)